Privify

A Computer Vision Pipeline for GDPR-Compliant Face Anonymization

Marco Contin
Epicode Institute of Technology
s00006824@epicode.edu.mt

2026-06-26

Abstract Privify is a Python-based computer vision pipeline that detects and anonymizes faces in video footage to support GDPR compliance for small and medium enterprises operating CCTV systems. The system combines a fine-tuned YOLOv8n detector (mAP@50 = 0.937 on a single-class face dataset) with a configurable Gaussian blur anonymizer. This report describes the pipeline architecture, training methodology, empirical evaluation, and the domain-shift failure mode observed when deploying the in-distribution-optimized detector on street-level CCTV scenes. This is a concrete illustration of the metric-vs-behavior gap in applied ML.

Contents

1 Problem Statement

The widespread adoption of video surveillance among Italian small and medium enterprises (SMEs) has created a regulatory paradox: the same technology that protects people and assets exposes their data to significant compliance risks under the General Data Protection Regulation (GDPR, EU 2016/679). A CCTV camera in a retail store, an office, or a logistics site daily captures dozens of recognizable faces (employees, customers, visitors) that constitute personal data under Article 4 of the GDPR. It is worth specifying that CCTV footage does not automatically qualify as biometric data protected under Article 9: this category applies only when images are processed through facial recognition systems for the unique identification of a natural person. However, even within the standard personal data regime, the data controller is required to adopt technical and organizational measures proportionate to risk, in accordance with the principles of accountability (Art. 5.2) and privacy by design (Art. 25). When recordings must be retained beyond the ordinary term, shared with third parties (insurance companies, law enforcement, audits), or even partially published, these measures must concretely reduce the identifiability of the subjects captured. For the average Italian SME, which typically lacks a dedicated Data Protection Officer and a substantial IT budget, this requirement often translates into inaction: videos are deleted for safety or, worse, retained in violation, with exposure to the sanctions provided under Article 83.5 of the GDPR, up to 20 million euros or 4% of annual worldwide turnover.

Existing market solutions polarize into two segments that leave the intermediate space uncovered. On one side, solutions such as Brighter AI or Pimloc Secure Redact offer production-grade pipelines targeting the enterprise segment, typically with pricing that is not publicly listed or tied to consumption/volume models. This commercial orientation makes them inaccessible to IT consultants serving SMEs as an off-the-shelf purchase. On the other side, manual solutions based on professional video editors (Adobe Premiere, DaVinci Resolve) require expert operators and production time linear in the number of frames, economically prohibitive on realistic CCTV volumes. Numerous open-source face blurring projects also exist (typically educational notebooks or single-file scripts on GitHub), but they rarely offer the level of engineering rigor required for professional deployment: automated testing, continuous integration, explicit versioning of models and datasets, traceable architectural decisions. The opportunity space for IT consultants working with Italian SMEs is therefore a system that combines accessibility (open-source, self-hostable) with engineering maturity (auditable, tested, documented), characteristics that the GDPR itself rewards through the accountability principle of Article 5.2.

Privify is designed to occupy this space. It is an open-source computer vision pipeline that automatically detects faces in CCTV footage and applies a configurable Gaussian blur as a visual redaction measure. Whether this operation qualifies as anonymization in the strict GDPR sense (Recital 26) depends on contextual parameters such as blur intensity, frame resolution, and the presence of other identifiers in the scene. Recent research has demonstrated that Gaussian blur can be partially attacked through deblurring and re-identification techniques [citation placeholder: gaussian blur deanonymization paper]; the system is therefore designed to allow the data controller to calibrate parameters in an informed manner and to document the technical choice adopted. The system combines a YOLOv8n detector fine-tuned on a single-class face dataset (mAP@50 = 0.937 on the validation set) with a stateless OpenCV-based anonymizer. The entire pipeline is written in Python, accompanied by an automated test suite, continuous integration on GitHub Actions, and Architectural Decision Records documenting every technical choice. The target deployment is the IT consultant working with Italian SMEs: the system must run on a laptop or a small server without dedicated GPU, process archived video in times acceptable for batch use, and produce redacted output that the data controller can legitimately retain or share. The extension of the detector to the license plate class, part of Privify's original positioning for the Italian CCTV context (drive-through, private parking, commercial entrances), is planned as future work for v0.2.

This report documents the development of Privify in its first iteration (v0.1), from the initial architectural choice through the detector fine-tuning and the empirical evaluation on test videos. The main contribution is not algorithmic novelty: YOLOv8 and Gaussian blur are established techniques. It is instead the engineering quality of the integration and the transparency with which both the successes (high detection

metrics in the in-distribution domain) and the limits (qualitative failure on street-level CCTV scenes, due to a textbook case of domain shift between training and deployment) are documented. The latter evidence, far from constituting a defect to conceal, represents the true didactic contribution of the project: a concrete demonstration of the gap between performance metric and qualitative behavior that characterizes applied machine learning, and an explicit roadmap to address it in subsequent iterations.

2 Methodology

2.1 Pipeline Architecture

The pipeline's entry point is the `process_video` function in `src/pipeline.py`, with the following signature:

```
def process_video(
    input_path: Path,
    output_path: Path,
    *,
    mode: Literal["face", "person_upper", "person_full"] | None = None,
    detector: Detector | None = None,
    anonymizer: Anonymizer | None = None,
) -> ProcessingStats:
```

The use of the `*` marker enforces that `mode`, `detector`, and `anonymizer` are provided as keyword arguments, preventing positional errors. The function supports two alternative usage modalities: preset configuration via the `mode` parameter (which automatically selects the appropriate detector/anonymizer pair, described in §2.4), or explicit dependency injection of `Detector` and `Anonymizer` instances for advanced scenarios. The internal resolver's validation ensures that at least one of the two modalities is provided, raising `ValueError` otherwise.

The main loop operates on frames read via `cv2.VideoCapture`, invoking for each `detector.detect(frame)` and subsequently `anonymizer.anonymize(frame, detections)`. The anonymized frame is written through `cv2.VideoWriter` with the `mp4v` codec, chosen for its availability in any OpenCV build without external dependencies (ffmpeg, H.264). Resource management adopts the `try`/`finally` pattern to guarantee the release of `VideoCapture` and `VideoWriter` even in case of exceptions during processing.

The final result is encapsulated in the frozen dataclass `ProcessingStats`, which exposes the total frames processed, the cumulative number of detections, the wall-clock time of processing, and the derived property `fps_processed`. The dataclass's immutability reflects the nature of the result as a value object at the end of an execution.

The architecture adopts three main design patterns. The Wrapper pattern isolates external dependencies (Ultralytics YOLO for the detector, OpenCV for the anonymizer) behind stable interfaces, allowing for potential future backend swaps (for example to ONNX Runtime for inference optimizations). Lazy loading of the YOLO model defers weight loading until the first call to `detect()`, making `Detector` instantiation free and enabling fast unit tests with mocks. Dependency injection centralizes in the `_resolve_components` resolver the construction of the detector/anonymizer pair based on `mode`, while maintaining the possibility of manual injection for testing and advanced use cases.

The entire implementation is covered by an automated test suite of 68 tests distributed across 6 test modules, executed in continuous integration on GitHub Actions. The four substantial modules in `src/` (`detector.py`, `anonymizer.py`, `pipeline.py`, `training.py`) amount to approximately 590 effective lines of code, excluding comments and blank lines.

2.2 Detection Models

The system supports three configurable detection models, each trained on a different image distribution. All are variants of Ultralytics' YOLOv8n architecture, chosen for its balance of accuracy and footprint:

72 layers, approximately 3 million parameters, 8.1 GFLOPs, and a disk footprint of around 6 MB. This choice is motivated by the declared deployment target (Italian SMEs without dedicated GPUs), which requires inference executable on CPU or entry-level consumer GPUs.

The first model, `face-detector-v0.1`, is obtained by fine-tuning YOLOv8n on Mohamed Traore's Face Detection dataset (Roboflow Universe, CC BY 4.0 license), composed of 1558 images of predominantly frontal close-up faces. This model is published as a GitHub Release with verified SHA-256, and is the default of the `Detector` wrapper when instantiated without arguments.

The second model, `face-detector-v0.2`, is a second iteration trained on a 3000-image subset extracted from WIDER FACE, biased toward the "hard" bin (faces of area smaller than 0.5% of the image, a distribution that reflects the CCTV deployment target). The rationale for this iteration is documented in §4 (Failure Analysis). This model is also published as a GitHub Release with SHA-256 integrity verification.

The third "model" is not a fine-tuning but the official Ultralytics pre-trained checkpoint `yolov8n.pt`, trained on COCO (80 classes). It is used by filtering detections to the `person` class only, and provides a generic person detector as a component of the hybrid modes described in §2.4. It is not versioned in the repo (Ultralytics downloads it automatically on first use), and is cached locally in `models/yolov8n.pt`.

The `Detector` wrapper accepts four optional parameters: `model_path` (path to the `.pt` file, defaulting to `face-detector-v0.1.pt`), `conf_threshold` (minimum confidence threshold, default 0.25, validated in (0, 1]), `device` (CPU or CUDA, optional), and `class_filter` (list of class names to retain in the result, optional). Detections are returned as instances of the frozen dataclass `Detection`, exposing bounding box (a tuple of four integers in pixel coordinates), confidence, class identifier, and class name.

2.3 Anonymization

The anonymization of detected regions is implemented in `src/anonymizer.py` through the stateless `Anonymizer` wrapper, which encapsulates the `cv2.GaussianBlur` algorithm from OpenCV. The choice of Gaussian blur (over pixelation or face replacement) is motivated by three factors. The first is visual coherence: blur preserves the subject's motion and silhouette, keeping the output video usable for non-identifying analysis (counts, flows, aggregate behaviors). The second is the absence of external dependencies beyond OpenCV, simplifying deployment. The third is documentary: compared to pixelation, whose reversibility under controlled conditions is documented in the literature (McPherson, Shokri, Shmatikov, 2016), Gaussian blur does not offer guarantees of irreversible anonymization in an absolute sense. The system is therefore designed to allow the data controller to calibrate parameters in an informed manner.

The `Anonymizer` constructor accepts four parameters: `blur_kernel_size` (positive odd integer, default 51), `min_bbox_size` (minimum width/height threshold for applying blur, default 4 pixels), `box_mode` (literal `"full"` or `"upper"`, default `"full"`), and `upper_fraction` (fraction of the upper portion of the bounding box to blur in `"upper"` mode, default 0.30, validated in (0, 1]). Parameter validation occurs in `__init__` with explanatory error messages.

The flow of the `anonymize(frame, detections)` method is non-destructive: it operates on a copy of the original frame, and returns immediately if no detections are present. For each detection, the bounding box coordinates are clamped to the frame edges; if the resulting crop dimension is smaller than `min_bbox_size`, the detection is silently skipped (avoiding artifacts on detections of negligible size). In `"full"` mode, the entire bounding box is replaced with its blurred version. In `"upper"` mode, only the upper strip is blurred, with height computed as `int(roi_h * upper_fraction)` and a guaranteed minimum of one pixel to avoid degeneracies. The algorithm applies `cv2.GaussianBlur` with a square kernel of side `blur_kernel_size` and sigma automatically derived by OpenCV from the kernel size.

The module's test suite (10 tests in `tests/test_anonymizer.py`) covers the main edge cases: empty detections, bounding boxes partially outside the frame, dimensions below threshold, constructor parameter

validation, and behavioral equivalence between `"full"` mode and the pre-parameter behavior (regression test).

2.4 Hybrid Detection Strategy

The pipeline supports three preset anonymization modes, accessible via the `mode` parameter of `process_video`. The motivation for this architectural choice is documented in §4 (Failure Analysis) and is rooted in an empirical measurement: the fine-tuned face detector v0.1 exhibits a recall of 17.2% on the CCTV deployment target (10 street-level frames, manually annotated ground truth), while the COCO person baseline reaches 80.4% on the same target. The 63 percentage point difference quantifies a structural gap between training domain and deployment domain, which requires architectural mitigation in addition to the algorithmic one (§4.3).

The `face` mode adopts the fine-tuned face detector (`face-detector-v0.1` by default, replaceable with v0.2) and applies blur to the entire facial bounding box. It is the suitable modality for close-range scenarios where the face is visible and of significant size (intercoms, reception desks, public-facing counters).

The `person_upper` mode adopts the COCO baseline with filtering to the `person` class, and applies blur to the upper strip of the person's bounding box (30% of the height), as a proxy for the head-shoulder region. This modality prioritizes coverage over anatomical precision: in CCTV street-level scenes where the face is often reduced to a few pixels, detecting the whole person has higher recall than detecting the face, and blurring the upper region protects identifiability without occluding the entire subject.

The `person_full` mode adopts the same person detector but applies blur to the entire person bounding box, providing the most conservative anonymization for high-sensitivity scenarios (video publication, sharing with non-audited third parties). The trade-off is the loss of contextual information about subject movement and activity.

The choice among the three modes is therefore a policy decision left to the data controller, and the keyword-only nature of the `mode` parameter forces the selection to be explicit in calling code, preventing unintended implicit configurations.

2.5 Training Setup

The fine-tuning of the two face detection models is implemented in the `src/training.py` module, which exposes the function `train_face_detector(config: TrainingConfig) -> TrainingResult`. The configuration is encapsulated in the frozen dataclass `TrainingConfig`, which requires the path to the dataset's `data.yaml` file and the output directory, and accepts as optional parameters the base model (`yolov8n.pt` by default, with `yolov8s.pt` as an alternative), the number of epochs (50), the image size (640), the batch size (16), and the device (`cuda`). Validation in `__post_init__` verifies the existence of the dataset file and the positive values of integer parameters.

The training invocation delegates to Ultralytics' `model.train(...)` API without specifying optimizer or learning rate, leaving the automatic selection active (`optimizer='auto'`, which in Ultralytics 8.3 resolves to AdamW for small datasets). This choice is functional to experimental discipline: the same function and the same hyperparameters are used for both training runs (v0.1 and v0.2), with the only difference being the `dataset_yaml_path` passed in `TrainingConfig`. The controllability of the variable (the dataset) is therefore structurally guaranteed by the code design: the training notebook is parameterized by a single `DATASET_VERSION` variable read at the top, and no conditional logic on the version exists in `src/training.py`.

The results are encapsulated in the frozen dataclass `TrainingResult`, which exposes the path to the `best.pt` weights, the final metrics (precision, recall, mAP@50, mAP@50-95 on the validation set), and the execution time. Both training runs were performed on Google Colab with T4 GPU runtime; the weights and training curves are published as GitHub Releases for reproducibility.

3 Experimental Results

TODO: metriche del modello fine-tuned, training curves, qualitative evaluation, comparison vs COCO baseline. ~2 pagine.

3.1 Quantitative Metrics

3.2 Training Convergence

3.3 Qualitative Evaluation

4 Failure Analysis

TODO: domain shift documentato, root cause analysis, mitigation roadmap. ~1.5 pagine. SEZIONE FORTE — qui c'è il vero contributo narrativo del progetto.

4.1 Domain Shift on Street-Level CCTV

4.2 Root Cause Analysis

4.3 Mitigation Roadmap

5 Ethical Considerations

TODO: privacy by design, GDPR principle of minimization vs effectiveness, biometric quasi-identifiers, dataset licensing (CC BY 4.0), model licensing (AGPL-3.0), responsible deployment. ~1.5 pagine.

5.1 Privacy by Design

5.2 GDPR Alignment

5.3 Dataset and Model Licensing

5.4 Responsible Deployment Considerations

6 Conclusions

TODO: ricap, lessons learned, future work. ~0.5 pagine.

7 References

TODO: bibliografia minima (3-6 references). Citare YOLOv8 paper, McPherson 2016 sulla reversibilità di pixelation, GDPR text, Roboflow dataset attribution, eventuali altri paper letti.